

Chapter 4

Decision Boundaries and Perceptron Learning

1. Introduction

What is Classification?

In artificial intelligence and machine learning, classification is the task of assigning a piece of data into a specific group or class. The computer looks at information about an item and decides which category it belongs to.

For example, when a computer looks at an email, it decides if the email belongs to the class 'Spam' or the class 'Not Spam'. When a bank system looks at a loan application, it decides if the application belongs to 'Approved' or 'Rejected'.

Why Machines Need Decision Boundaries

A computer cannot rely on human intuition to make decisions. It requires precise mathematical rules to separate one category from another. To separate different groups of data, a machine constructs a separating line, surface, or boundary in mathematical space. This dividing line is known as a decision boundary.

Without a decision boundary, a machine learning model cannot make predictions on new, unseen data. The decision boundary serves as the absolute geometric fence that splits space into distinct prediction regions.

Binary Classification

Binary classification is a specific type of classification where there are exactly two possible categories or choices. The prefix 'bi-' means two. In binary classification, the machine chooses between two mutually exclusive outcomes.

In artificial neural networks, these two choices are usually represented mathematically using numbers:

- **Positive Class:** Represented as +1 (or 1).
- **Negative Class:** Represented as -1 (or 0).

Real-Life Examples

Binary classification is widely used across technology, healthcare, finance, and security. Table 1.1 illustrates common real-life binary classification tasks, their inputs, and their corresponding classes.

Email Filtering	Word frequency, Sender reputation	Legitimate Email	Spam Email
Medical Diagnosis	Blood pressure, Tumor size	Benign (Healthy)	Malignant (Sick)
Banking Fraud	Transaction amount, Location	Normal Transaction	Fraudulent Action
Academic Admission	GPA test score, Essay score	Rejected	Admitted

2. Decision Boundary

Definition

A decision boundary is a geometric hypersurface that partition a feature space into two or more regions, where each region corresponds to a specific class label predicted by the classification algorithm.

Simple Explanation

Imagine drawing a line on a piece of paper. On the left side of the line, you place blue dots. On the right side of the line, you place red dots. The line you drew is the decision boundary. If a new dot appears on the left side, you predict it is blue. If a new dot appears on the right side, you predict it is red.

Mathematical Explanation

Mathematically, a decision boundary is the exact set of points in input space where the machine's predictive output is completely neutral or ambiguous. For a model that uses a weighted sum function z , the decision boundary occurs precisely where $z = 0$.

Geometric Explanation

Geometrically, the decision boundary divides an n -dimensional space into two separate regions. Points lying on one side of the boundary satisfy $z > 0$, while points lying on the opposite side satisfy $z < 0$.

Decision Regions

A decision region is a subset of the input space where all points are assigned to the same class category. A binary perceptron creates exactly two decision regions:

- 1. Positive Decision Region:** The collection of all input vectors where the model predicts $+1$.
- 2. Negative Decision Region:** The collection of all input vectors where the model predicts -1 .

Hyperplane

A hyperplane is a flat geometric subspace whose dimension is exactly one less than the dimension of its ambient space. If the input space has n dimensions, the decision boundary is an $(n - 1)$ -dimensional hyperplane.

Line vs. Plane vs. Hyperplane

The terminology changes based on how many input variables (features) the problem contains. Table 2.1 summarizes how the dimension of the input space determines the geometry of the decision boundary.

Input (n)	Features	Space Dimension	Boundary Geometry	Boundary Dimension
1	Feature (x1)	1D Space (Line)	Point	0D (Point)
2	Features (x1, x2)	2D Space (Plane)	Straight Line	1D (Line)
3	Features (x1, x2, x3)	3D Space (Volume)	Flat Surface (Plane)	2D (Plane)
n	Features (x1..xn)	nD Space	Hyperplane	(n - 1)D Hyperplane

3. Mathematical Foundation

To understand how a perceptron makes decisions, we must break down its mathematical internal structure. The perceptron processes input signals using components called inputs, weights, biases, weighted sums, and activations.

Component Breakdown

Input (x): The raw quantitative data values fed into the model from external observations.

Feature: An individual measurable property or dimension of the data (e.g., height, weight, temperature).

Weight (w): A trainable parameter that represents the relative strength, importance, or influence of a specific feature in determining the output.

Bias (b): A trainable parameter that acts as an adjustable offset. It shifts the activation threshold independently of feature values.

Weighted Sum (z): The total scalar value calculated by multiplying each input feature by its corresponding weight and adding the bias.

Activation Function $f(z)$: A mathematical rule that converts the continuous weighted sum z into a final discrete class output.

Prediction (y_{hat}): The final output category computed by the perceptron for a given input observation.

The Perceptron Core Equations

The continuous weighted sum z is calculated using Equation (3.1):

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \left(\sum w_i x_i \right) + b \quad (3.1)$$

Where every symbol is defined as follows:

- z : The linear combination score (weighted sum).
- x_i : The i -th input feature value.
- w_i : The weight assigned to the i -th input feature.
- b : The scalar bias term.
- n : The total number of input features.

The output prediction y_{hat} is calculated using the step activation function in Equation (3.2):

$$\begin{aligned} y_{\text{hat}} &= +1 \text{ if } z \geq 0 \\ y_{\text{hat}} &= -1 \text{ if } z < 0 \end{aligned} \quad (3.2)$$

4. Geometry of Decision Boundary

The Boundary Equation

For a two-dimensional feature space with features x_1 and x_2 , the weighted sum equation is $z = w_1x_1 + w_2x_2 + b$. Because the boundary occurs where the perceptron is perfectly neutral ($z = 0$), the algebraic equation of the decision boundary is Equation (4.1):

$$w_1x_1 + w_2x_2 + b = 0 \quad (4.1)$$

Why This Equation Forms a Line

Equation (4.1) is a linear equation in two variables. If we map x_1 to the standard horizontal axis (x) and x_2 to the standard vertical axis (y), we can rearrange Equation (4.1) into the standard slope-intercept form ($y = mx + c$):

$$w_1x + w_2y + b = 0$$

$$w_2y = -w_1x - b$$

$$y = \left(-w_1 / w_2 \right) x + \left(-b / w_2 \right) \quad (4.2)$$

Equation (4.2) proves mathematically that the decision boundary is a straight line with:

- Slope (m) = $-w_1 / w_2$

- Vertical Intercept (c) = $-b / w_2$

Positive and Negative Sides

The decision boundary line splits the 2D plane into two distinct half-planes:

- Positive Side: The set of points (x_1, x_2) where $w_1x_1 + w_2x_2 + b > 0$. The perceptron outputs +1.
- Negative Side: The set of points (x_1, x_2) where $w_1x_1 + w_2x_2 + b < 0$. The perceptron outputs -1.

Weight Vector and Normal Vector

The weight vector $w = [w_1, w_2]^T$ points in the direction of maximum increase of the weighted sum z . Crucially, the weight vector is geometrically orthogonal (perpendicular) to the decision boundary line. Therefore, the weight vector serves as the normal vector to the decision boundary line, always pointing directly into the positive decision region.

Distance Intuition

The perpendicular distance d from any arbitrary data point (x_1^*, x_2^*) to the decision boundary line is given by Equation (4.3):

$$d = |w_1x_1^* + w_2x_2^* + b| / \sqrt{w_1^2 + w_2^2} \quad (4.3)$$

5. Effect of Weights

The weights w_1 and w_2 govern the orientation and slope of the decision boundary.

Positive vs. Negative Weights

If a feature has a positive weight, increasing that feature's value increases the total weighted sum z , making the model more likely to predict +1. Conversely, if a feature has a negative weight, increasing that feature's value decreases z , pushing the prediction toward -1.

Magnitude: Large vs. Small Weights

The magnitude of a weight determines how sensitive the decision boundary is to changes in that feature. A large absolute weight value means a small change in that input feature causes a drastic change in the output.

Weight Rotation Effect

Changing the ratio between w_1 and w_2 alters the slope $m = -w_1/w_2$. Therefore, adjusting weights rotates the decision boundary around its axis.

6. Effect of Bias

Why Bias Exists

Without a bias term ($b = 0$), the decision boundary equation simplifies to $w_1x_1 + w_2x_2 = 0$. Notice that if $x_1 = 0$ and $x_2 = 0$, the equation $w_1(0) + w_2(0) = 0$ is always true regardless of the values of w_1 and w_2 . This means a perceptron without a bias term is forced to draw its decision boundary directly through the origin $(0, 0)$.

Decision Boundary With vs. Without Bias

Adding the bias term b releases the decision boundary from the origin, allowing it to shift parallelly across the feature space.

Bias Shifting Effect

Changing the value of b shifts the decision boundary without changing its slope or orientation:

- Increasing b shifts the boundary line away from the positive region.
- Decreasing b shifts the boundary line toward the positive region.

GEOMETRIC PRINCIPLE: Bias does not rotate the boundary line. Bias only translates (shifts) the line in space. Weights control the angle (rotation), while bias controls the position (translation).

7. Decision Boundary Examples

To gain clear intuition, let us examine six explicit numerical examples. In all examples, we treat x_1 as x and x_2 as y .

Example 1: Basic Boundary ($x + y = 4$)

- Perceptron Parameters: $w_1 = 1$, $w_2 = 1$, $b = -4$
- Desmos Equation: $x + y - 4 = 0$
- Sample Data Points:
 - Point A (3, 3): $z = (1)(3) + (1)(3) - 4 = +2$. Prediction: +1 (Positive Region)
 - Point B (1, 1): $z = (1)(1) + (1)(1) - 4 = -2$. Prediction: -1 (Negative Region)
 - Point C (2, 2): $z = (1)(2) + (1)(2) - 4 = 0$. Lies exactly on the decision boundary.
- Desmos Observation: Students will see a straight line passing through (4,0) and (0,4) with slope $m = -1$.

Example 2: Steep Boundary Line ($2x + y - 5 = 0$)

- Perceptron Parameters: $w_1 = 2$, $w_2 = 1$, $b = -5$
- Desmos Equation: $2x + y - 5 = 0$
- Sample Data Points:
 - Point A (3, 1): $z = 2(3) + 1(1) - 5 = +2$. Prediction: +1
 - Point B (1, 1): $z = 2(1) + 1(1) - 5 = -2$. Prediction: -1
- Desmos Observation: The weight $w_1 = 2$ makes the line steeper (slope $m = -2$) compared to Example 1.

Example 3: Positive Slope Boundary ($3x - 2y + 1 = 0$)

- Perceptron Parameters: $w_1 = 3$, $w_2 = -2$, $b = 1$
- Desmos Equation: $3x - 2y + 1 = 0$
- Sample Data Points:
 - Point A (0, 2): $z = 3(0) - 2(2) + 1 = -3$. Prediction: -1
 - Point B (2, 0): $z = 3(2) - 2(0) + 1 = +7$. Prediction: +1
- Desmos Observation: Because w_2 is negative (-2), the slope $m = -3/(-2) = +1.5$ is positive.

Example 4: Boundary Through Origin ($x - y = 0$)

- Perceptron Parameters: $w_1 = 1$, $w_2 = -1$, $b = 0$
- Desmos Equation: $x - y = 0$
- Sample Data Points:
 - Point A (2, 1): $z = 1(2) - 1(1) + 0 = +1$. Prediction: +1
 - Point B (1, 2): $z = 1(1) - 1(2) + 0 = -1$. Prediction: -1
- Desmos Observation: Because $b = 0$, the line passes directly through the origin (0,0).

Example 5: Effect of Changing Only Bias

- Baseline Equation: $2x + 2y - 4 = 0$ ($b = -4$)
- Shifted Line A: $2x + 2y - 8 = 0$ ($b = -8$)
- Shifted Line B: $2x + 2y + 2 = 0$ ($b = +2$)
- Desmos Observation: Plotting all three lines simultaneously shows three parallel lines. Changing bias shifts the line parallelly without changing its rotation.

Example 6: Effect of Changing Only Weights

- Baseline Equation: $1x + 1y - 4 = 0$ ($w_1=1$, $w_2=1$)

- Rotated Line A: $3x + 1y - 4 = 0$ ($w_1=3, w_2=1$)
- Rotated Line B: $0.5x + 1y - 4 = 0$ ($w_1=0.5, w_2=1$)
- Desmos Observation: All three lines intersect the vertical y-axis at the exact same point (0, 4), but rotate at different angles.

8. Linearly Separable Data

Definition

A dataset is said to be linearly separable if there exists at least one single straight line (or hyperplane in higher dimensions) that completely separates all positive instances from all negative instances without making a single error.

Examples of Linearly Separable Functions

Logical operations like AND and OR are classic examples of linearly separable data. Table 8.1 shows the AND logic gate dataset, which can be perfectly separated by the decision boundary line $x_1 + x_2 - 1.5 = 0$.

Input Feature x1	Input Feature x2	Weighted Sum z	Target Class Output
0	0	-1.5	-1
0	1	-0.5	-1
1	0	-0.5	-1
1	1	+0.5	+1

Why the Perceptron Works on Linearly Separable Data

The fundamental architecture of a perceptron relies on a linear weighted sum $z = \sum w_i x_i + b$. Because this mathematical structure is inherently linear, a standard single-layer perceptron is fully capable of solving any linearly separable problem.

9. Non-Linearly Separable Data

Definition

A dataset is non-linearly separable if it is impossible to draw a single straight line (or flat hyperplane) that separates all positive data points from all negative data points without error.

The XOR Problem

The Exclusive OR (XOR) logical function is the most famous example of a non-linearly separable problem. Table 9.1 displays the truth table for the XOR gate.

Input x1	Input x2	Target Class Output
0	0	-1
0	1	+1
1	0	+1
1	1	-1

Notice that positive points (+1) are located at diagonally opposite corners (0,1) and (1,0), while negative points (-1) are located at (0,0) and (1,1). It is geometrically impossible to draw a single straight line that separates (0,1) and (1,0) from (0,0) and (1,1).

Other Non-Linear Patterns

- Concentric Circles: Positive points form a small inner circle, while negative points form a larger surrounding ring.
- Spiral Data: Positive and negative points intertwine in two interlaced spiral arms.

10. Perceptron Learning

Perceptron learning is the automated process by which the model adjusts its weights and bias based on training data until it correctly classifies all training instances.

Terminology

- Training Data: A collection of input-target pairs used to train the model.
- Target (y): The true, correct class label for a given input instance.
- Error (e): The numerical difference between the target label and predicted label: $e = y - \hat{y}$.
- Epoch: One full pass through the entire training dataset.
- Stopping Criteria: The condition under which training stops (e.g., zero error across an epoch).

Complete Training Cycle

1. Initialize weights and bias to small random numbers or zeros.
2. Select a sample pair (x, y) from the training set.
3. Compute the predicted output $\hat{y}$ using the current weights and bias.

4. Calculate error $e = y - y_{\text{hat}}$.
5. Update weights and bias based on the error.
6. Repeat for all training samples.
7. Check stopping criteria. If errors remain, start a new epoch.

11. Perceptron Learning Rule

Learning Rate

The learning rate, denoted by the Greek letter eta (η), is a small positive constant hyperparameter (typically between 0.001 and 1.0) that controls how drastically weights change during each update.

Weight & Bias Update Formulas

The weight and bias update equations are given by Equations (11.1) and (11.2):

$$\mathbf{w}_i(\text{new}) = \mathbf{w}_i(\text{old}) + \Delta \mathbf{w}_i \quad \text{where} \quad \Delta \mathbf{w}_i = \eta \cdot (y - y_{\text{hat}}) \cdot \mathbf{x}_i \quad (11.1)$$

$$\mathbf{b}(\text{new}) = \mathbf{b}(\text{old}) + \Delta \mathbf{b} \quad \text{where} \quad \Delta \mathbf{b} = \eta \cdot (y - y_{\text{hat}}) \quad (11.2)$$

Mathematical & Geometric Intuition

If the prediction is correct ($y_{\text{hat}} = y$), the error ($y - y_{\text{hat}}$) is 0, so $\Delta \mathbf{w} = 0$ and $\Delta \mathbf{b} = 0$. No update occurs. If the target is +1 but the model predicted -1, error is +2. The rule adds a portion of the input vector $\mathbf{x}$ to the weight vector $\mathbf{w}$, rotating the decision boundary line toward the misclassified positive point.

12. Complete Worked Example

Let us manually train a perceptron to learn the **AND** logic gate from scratch.

Given:

- Learning Rate: $\eta = 0.1$
- Initial Weights: $w_1 = 0.0, w_2 = 0.0$
- Initial Bias: $b = 0.0$

The training data is:

<i>Input (x_1, x_2)</i>	<i>Target (y)</i>
$(0,0)$	-1
$(0,1)$	-1
$(1,0)$	-1
$(1,1)$	+1

Epoch 1

Sample 1

Input: (0,0)

Target: -1

$$z = (0)(0) + (0)(0) + 0 = 0$$

Prediction:

$$\hat{y} = +1$$

Error:

$$e = y - \hat{y} = -1 - (+1) = -2$$

Update

$$\Delta w_1 = 0.1(-2)(0) = 0$$

$$\Delta w_2 = 0.1(-2)(0) = 0$$

$$\Delta b = 0.1(-2) = -0.2$$

Updated Parameters

- $w_1 = 0.0$
- $w_2 = 0.0$
- $b = -0.2$

Sample 2

Input: (0,1)

Target: -1

$$z = (0)(0) + (0)(1) - 0.2 = -0.2$$

Prediction:

$$\hat{y} = -1$$

Error:

$$e = -1 - (-1) = 0$$

Correct prediction.

No parameter update.

Sample 3

Input: (1,0)

Target: -1

$$z = (0)(1) + (0)(0) - 0.2 = -0.2$$

Prediction:

$$\hat{y} = -1$$

Error:

$$e = 0$$

Correct prediction.

No update.

Sample 4

Input: (1,1)

Target: +1

$$z = (0)(1) + (0)(1) - 0.2 = -0.2$$

Prediction:

$$\hat{y} = -1$$

Error

$$e = 1 - (-1) = 2$$

Update

$$\Delta w_1 = 0.1(2)(1) = 0.2$$

$$\Delta w_2 = 0.1(2)(1) = 0.2$$

$$\Delta b = 0.1(2) = 0.2$$

Updated Parameters

- $w_1 = 0.2$
- $w_2 = 0.2$
- $b = 0.0$

Epoch 2

Sample 1

Input: (0,0)

Target: -1

$$z = (0.2)(0) + (0.2)(0) + 0 = 0$$

Prediction:

$$\hat{y} = +1$$

Error

$$e = -2$$

Update

$$\begin{aligned}\Delta w_1 &= 0 \\ \Delta w_2 &= 0 \\ \Delta b &= -0.2\end{aligned}$$

Updated Parameters

- $w_1 = 0.2$
- $w_2 = 0.2$

- $b = -0.2$

Sample 2

Input: (0,1)

Target: -1

$$z = (0.2)(0) + (0.2)(1) - 0.2 = 0$$

Prediction:

$$\hat{y} = +1$$

Error

$$e = -2$$

Update

$$\Delta w_1 = 0$$

$$\Delta w_2 = -0.2$$

$$\Delta b = -0.2$$

Updated Parameters

- $w_1 = 0.2$
- $w_2 = 0.0$
- $b = -0.4$

Sample 3

Input: (1,0)

Target: -1

$$z = (0.2)(1) + (0)(0) - 0.4 = -0.2$$

Prediction:

$$\hat{y} = -1$$

Error

$$e = 0$$

No update

Sample 4

Input: (1,1)

Target: +1

$$z = (0.2)(1) + (0)(1) - 0.4 = -0.2$$

Prediction

$$\hat{y} = -1$$

Error

$$e = 2$$

Update

$$\Delta w_1 = 0.2$$

$$\Delta w_2 = 0.2$$

$$\Delta b = 0.2$$

Updated Parameters

- $w_1 = 0.4$

- $w_2 = 0.2$
- $b = -0.2$

Training Summary

Epoch	Sample	Input	Target	z	Prediction	Error	Updated Weights	Updated Bias
1	1	(0,0)	-1	0.0	+1	-2	(0.0, 0.0)	-0.2
1	2	(0,1)	-1	-0.2	-1	0	(0.0, 0.0)	-0.2
1	3	(1,0)	-1	-0.2	-1	0	(0.0, 0.0)	-0.2
1	4	(1,1)	+1	-0.2	-1	+2	(0.2, 0.2)	0.0
2	1	(0,0)	-1	0.0	+1	-2	(0.2, 0.2)	-0.2
2	2	(0,1)	-1	0.0	+1	-2	(0.2, 0.0)	-0.4
2	3	(1,0)	-1	-0.2	-1	0	(0.2, 0.0)	-0.4
2	4	(1,1)	+1	-0.2	-1	+2	(0.4, 0.2)	-0.2

Final Parameters after Epoch 2

- **Weights:** (0.4, 0.2)
- **Bias:** -0.2

13. Complete Perceptron Learning Algorithm

The standard Perceptron Learning Algorithm follows a clear sequence of operations.

PSEUDOCODE:

ALGORITHM PerceptronTrain($X, Y, \text{learning_rate}, \text{max_epochs}$):

Initialize weights $w = [0, 0, \dots, 0]$

Initialize bias $b = 0$

For epoch from 1 to max_epochs :

$\text{total_errors} = 0$

 For each sample x_i with target y_i in dataset:

$z = \text{dot_product}(w, x_i) + b$

$y_{\text{hat}} = \text{step_function}(z)$

$\text{error} = y_i - y_{\text{hat}}$

 If $\text{error} \neq 0$:

$w = w + \text{learning_rate} * \text{error} * x_i$

$b = b + \text{learning_rate} * \text{error}$

$\text{total_errors} = \text{total_errors} + 1$

 If $\text{total_errors} == 0$:

 Break (Model converged)

Return w, b